

KOHLER Unified Enterprise AI Agent

Track 3 — Unified Enterprise AI Agent

Comprehensive Research, Prompt, System-Instruction & Workflow Documentation

Compiled from the six prior KOHLER research/development chats and the project artifacts recovered from the user's Library. This document intentionally excludes the current chat.

Prepared for repository documentation and evaluator review.
September 2026

1. Scope and Source Note

This dossier reconstructs the work completed across the six prior KOHLER chats: (1) project/Track selection and planning, (2) Phase 1 Enterprise Research, (3) Phase 2 Solution Design, (4) Phase 3 detailed agent/data/HITL design, (5) Phase 4 System Architecture, and (6) Phase 5 Prototype Implementation. The Phase 5 chat also included the readiness discussion for moving to Phase 6.

Important provenance: the recovered PROMPTS.md explicitly states that it is a reconstructed development record, not a verbatim export of the complete private conversation history. Therefore, this PDF preserves exact recovered prompt text where it exists and labels reconstructed prompt themes/system requirements rather than presenting them as verbatim conversation transcripts.

No content from the current chat is included in the six-chat research record. The current repository state is used only to document the implementation that resulted from that prior work.

Six-chat map

Chat	Primary work recovered
1 — Plan KOHLER AI Research	Track 3 selection, problem framing, required learning topics, initial architecture and differentiator.
2 — Phase 1 Enterprise Research	Enterprise AI patterns, orchestration role, security/governance, RAG, agents, tools, HITL and research areas.
3 — Phase 2 Solution Design	System components, agent responsibilities, retrieval/tool split, orchestration lifecycle, security and MVP scope.
4 — Phase 3	Detailed implementation design: agents, secure retrieval, data-entry workflows, Finance/Operations HITL, approvals and auditability.
5 — Phase 4	System architecture, end-to-end workflow, free/local technology choices, observability and governance.
6 — Phase 5	Prototype implementation: authentication, RBAC, RAG, agents, orchestrator, live data, HITL, audit logs, UI and testing.

2. Prior Chat 1 — Track 3 Selection & Project Framing

Project direction

The project was framed as a **Unified Enterprise AI Agent**: an intelligent front door and orchestration layer across fragmented enterprise knowledge, data, tools and specialist agents. The design was explicitly positioned as more than a generic chatbot.

Initial architecture

- User through Web/Teams/Mobile-style interface → unified/master agent for intent, planning and reasoning.
- Master agent → specialist agents such as Knowledge, Product, Sales, Supply Chain, Finance, HR and IT.
- Unstructured documents → RAG; structured/live data → APIs, SQL, connectors and tools.
- Cross-cutting controls → identity, RBAC, guardrails, HITL, audit and observability.

Core lifecycle patterns explored

Understand → Plan → Retrieve → Reason → Act → Verify

Intent → Retrieve → Select → Orchestrate → Execute → Verify → Respond

The project direction converged on a controlled enterprise AI layer that connects natural-language requests to authorized information and governed actions.

Why orchestration alone was not enough

The key differentiation question was: if every participant builds orchestration, what makes this implementation distinct? The answer developed during the research was to combine orchestration with **RBAC, permission-aware retrieval, live enterprise state, controlled actions, HITL approvals and auditability**.

Recovered development prompt themes

Help research and plan Track 3 – Unified Enterprise AI Agent, including the topics needed before implementation.

If every participant uses agent orchestration, what makes this project different?

3. Prior Chat 2 — Phase 1: Enterprise Research

Research objective

Phase 1 established the enterprise problem and the role of the unified agent. The core conclusion was that the platform should connect existing agents, enterprise data, APIs/tools and knowledge rather than attempting to replace every departmental system.

Security research

- Identity and RBAC are first-class controls.
- Permission-aware retrieval should occur before protected context reaches the model.
- Least privilege should limit data and actions by designation.
- High-impact actions should require human approval.
- Sensitive activity should be traceable through an audit trail.
- Prompt-injection and data-leakage risks require application-level guardrails.
- The system should fail safely rather than silently bypassing authorization.

Security traceability pattern

User → Request → Data → Agent → Tool → Action → Approval

Phase 1 conclusion

The prototype should demonstrate a vertical slice of the larger enterprise architecture: role-aware access, cross-department routing, RAG for enterprise knowledge, live structured data for current state, controlled actions and human approval.

Phase 1 prompt theme

Help research Track 3 – Unified Enterprise AI Agent and identify the enterprise architecture, security, RAG, agent, c

4. Prior Chat 3 — Phase 2: Solution Design

Solution boundary

Phase 2 translated the research into a concrete architecture and MVP. The solution was organized around architecture, agents, RAG, tools/APIs, orchestration, memory, security/governance and prototype scope.

Agent model

Component	Responsibility
Unified Orchestrator	Understand intent, identify departments/actions, route work and combine results.
HR Agent	HR policy, employee-record and manager/team-level requests subject to permissions.
Finance / Enterprise Agent	Finance and enterprise knowledge retrieval with role-aware filtering.
Operations Agent	Inventory, production and supplier/current-state operations requests.
RAG / Knowledge Base	Policies, procedures and other unstructured enterprise knowledge.
SQL / Live Data	Current employees, inventory, finance and operational state.
Action Layer	Validated database mutations after authorization and approval.
HITL / Approval	Human control for sensitive state changes.
Audit	Trace requester, action, approver, status and timestamp.

Central design principle

RAG answers knowledge questions; database tools answer live-state questions; HITL protects sensitive state changes.

Phase 2 prompt themes

Design a unified enterprise AI architecture with departmental agents, orchestration, RAG, permissions and one common
Combine RAG for policies/documents with live SQL data for current enterprise state and HITL for sensitive writes.

Autonomy model researched

- Answer — retrieve and explain.
- Recommend — analyze and propose.
- Prepare — draft an action for review.
- Execute — perform an authorized action.
- Autonomous — only for appropriately bounded low-risk tasks.

5. Prior Chat 4 — Phase 3: Detailed Solution & HITL Design

Permission-aware retrieval

The design required authorization to be decided by the application, not by the LLM. The model should reason only over context the backend has already authorized.

Prevent employees from retrieving finance, HR, or operations information they are not authorized to access.

This became the secure-search pattern: authenticate the user, resolve designation/permissions, enforce access before retrieval, then pass only authorized context into the agent/LLM.

Role-specific login

Create a login system where employee designation controls what information and actions the user can access.

The later prototype implemented company-email login, JWT access tokens and designation-based permissions. Roles included Employee, Manager, Finance Admin, HR Admin, Operations Manager, Operations Admin and GM.

Finance HITL

Sensitive finance modifications should create an approval request and execute only after GM approval.

The intended workflow was: request → permission check → approval request → GM review → execute → audit.

Operations HITL

Inventory/production changes should require GM approval and execute only after approval.

The same controlled-write pattern was applied to operational state. An Operations Admin can request a change, but the change is not committed until the GM approves it.

Role-specific data entry

Let authorized users submit new data through dashboard forms; validate it, route it through approval, and add approvals.

Audit design

Record requester, action, details, approver, status, agent/department, and timestamp.

6. Prior Chat 5 — Phase 4: System Architecture

Layered architecture

- Presentation layer: React UI for login, chat, data entry and approvals.
- API/security layer: FastAPI, authentication, RBAC and validation.
- Orchestration layer: intent understanding, decomposition, routing, execution, verification and response.
- Agent layer: HR, Finance/Enterprise and Operations specialists.
- Knowledge/data layer: RAG plus structured SQL data.
- Control layer: permissions, HITL, audit and observability.

Orchestrator lifecycle

Understand → Decompose → Route → Execute → Verify → Respond

Free/local constraint

A strict project constraint was established: **no paid software, paid APIs, subscriptions or credit-card-dependent services**. The chosen workflow therefore emphasized local/open tooling.

Layer	Chosen direction
Frontend	React + Vite
Backend	FastAPI / Python
Agent framework	LangChain / orchestration patterns
Local LLM	Ollama with a local model
RAG	ChromaDB / vector retrieval
Database	SQLite + SQLAlchemy for the local prototype
Embeddings	Sentence-transformers/local embeddings
Auth	JWT
HITL	Custom React/FastAPI approval workflow
Logging	Python/application audit logging

Approval record design

The architecture discussed an approval_requests/approval-record pattern containing requester, agent/action, affected data, risk/status information, timestamps and approver information.

Architecture workflow

Natural Language Request → Authentication → RBAC → Intent / Orchestrator → RAG or Database → Sensitive Action? →

7. Prior Chat 6 — Phase 5: Prototype Implementation

Implementation sequence

- Authentication and JWT login.
- Role/permission matrix.
- Secure RAG and document ingestion.
- HR Agent with role-aware policy/employee access.
- Operations Agent with live inventory access.
- Finance / Enterprise secure search.
- Intent and department detection.
- Unified multi-agent orchestrator.
- HITL approval creation and GM approval.
- Approved database actions.
- Audit logging.
- React login/dashboard integration.
- End-to-end validation.

Authentication system

The backend creates JWTs containing employee ID and designation. Protected endpoints use a FastAPI dependency to decode the token. The frontend login calls the authentication endpoint and uses the returned token for protected operations.

RBAC matrix

Role	Representative permissions
Employee	Own-department visibility
Manager	Department/team HR visibility
Finance Admin	Finance view/modify/approve and HR policy view
HR Admin	HR view, employee records and HR modification
Operations Manager	Operations, inventory and production view
Operations Admin	Operations/inventory/production view + modification
GM	Company/confidential view, cross-department view and approval authority

Secure RAG system instruction pattern

Use only the role-authorized context supplied by the application. Answer concisely and factually. Do not invent employee records or company information. This pattern was used to stop the local LLM from filling gaps with invented enterprise information.

HR Agent prompt/guardrails

Answer only from authorized HR context. Employee records and personal information require the appropriate employee-restricted context.

Operations Agent prompt/guardrails

Answer Operations questions using the live authorized inventory/operations context. For current quantities, use the current data.

8. Recovered Development Prompt Catalogue

The following catalogue is the recovered PROMPTS.md record. The original artifact labels it as a reconstructed development record rather than a verbatim transcript. These entries are therefore the authoritative recovered prompt themes from the project documentation.

1. Track 3 Research

Help research and plan Track 3 – Unified Enterprise AI Agent, including the topics needed before implementation.

2. Enterprise Architecture

Design a unified enterprise AI architecture with departmental agents, orchestration, RAG, permissions, and one common

3. Project Differentiation

If every participant uses agent orchestration, what makes this project different?

4. Role-Based Login

Create a login system where employee designation controls what information and actions the user can access.

5. Permission-Aware RAG

Prevent employees from retrieving finance, HR, or operations information they are not authorized to access.

6. Multi-Agent Routing

Detect whether a query belongs to HR, Finance, Operations, or general enterprise knowledge and route it to the correct

7. HR Agent

Build an HR agent for HR policies, employee records, and manager team-level information with permission checks.

8. Operations Agent

Build an Operations Agent for inventory and production questions with role permissions.

9. Finance HITL

Sensitive finance modifications should create an approval request and execute only after GM approval.

10. Operations HITL

Inventory/production changes should require GM approval and execute only after approval.

11. Audit Logging

Record requester, action, details, approver, status, agent/department, and timestamp.

12. GM Dashboard

Show Pending Approvals and Audit Logs only to the GM and allow the GM to review/approve requests.

13. Authentication Debugging

Fix the FastAPI JWT dependency/CORS/failed-fetch problem when the frontend calls protected endpoints.

14. Confidential Information

Ensure application-level RBAC is authoritative when the local LLM refuses an authorized confidential request.

15. Live Enterprise Data

Replace static values for current state with synthetic SQL enterprise data so approved changes are reflected in subse

16. Synthetic Dataset

Create realistic synthetic data for employees, HR, Finance, Operations, inventory, suppliers, and audit records.

17. Role-Specific Data Entry

Let authorized users submit new data through dashboard forms; validate it, route it through approval, and add approvals.

18. Final Hybrid Architecture

Combine RAG for policies/documents with live SQL data for current enterprise state and HITL for sensitive writes.

19. Evaluation Story

Present the project using approach and innovation; technical execution; user experience and feasibility; business success.

9. System Instructions & Guardrails Used in the Solution

The following are implementation-level system-instruction requirements recovered from the agent design and prototype. They are presented as guardrail specifications rather than claimed verbatim hidden prompts.

Authorization is authoritative

The application decides whether the user is authorized. The model must never grant, infer, expand, or override permissions.

No invention / hallucinated enterprise records

Use only supplied authorized enterprise context or live database results. Do not fabricate names, salaries, quantities, or other data.

Department isolation

Answer only the department-specific part of a delegated request. Do not use information from another department unless explicitly requested.

Live state precedence

When the question asks for current inventory or other mutable enterprise state, use the authorized live database results over static documentation.

Sensitive-write control

Never directly perform a sensitive mutation merely because the user requested it. Verify modification permission, create a record, and obtain approval.

Auditability

Record sensitive actions and approvals with requester, agent/department, action, details, approver, status and timestamp.

Safe failure

If authorization, required context, tool output or approval is missing, do not guess. Return a clear denial or unavailability message.

10. End-to-End Workflows

A. Read / knowledge request

User → JWT authentication → RBAC → intent/department detection → specialist agent → permission-aware RAG or author

B. Cross-department read

Mixed query → detect multiple departments → split into department-specific subqueries → HR/Finance/Operations agent

This was specifically implemented after testing showed that sending the full mixed query to every agent caused cross-domain answers. The orchestrator was changed to delegate department-specific subqueries.

C. Sensitive modification

User request → authenticate → permission check → create pending approval → GM reviews → approve/reject → if appro

D. Role-specific data entry

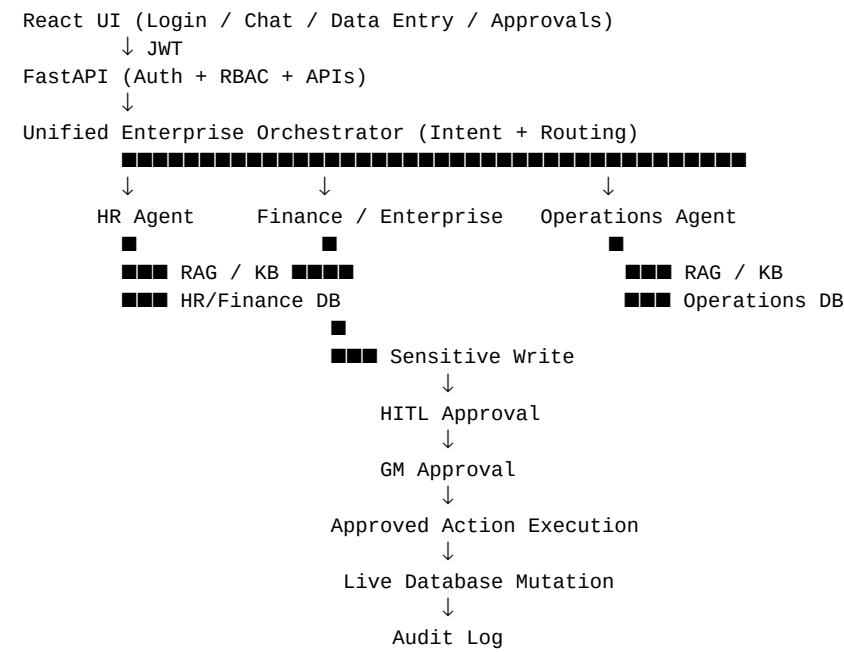
Authorized user → role-specific form → validation → permission check → approval request when sensitive → authoriz

E. Live inventory example

Operations Admin: "Increase Industrial Valves by 10 units"
↓
Permission check
↓
Pending OPERATIONS_MODIFICATION
↓
GM approval
↓
Operations action executes
↓
Inventory database updated
↓
Audit record written
↓
Next inventory query returns updated value

The tested prototype reached a live Industrial Valves quantity of 345 after approved modifications. The exact value is implementation state, not a seed-data claim.

11. Final Hybrid Architecture



RAG versus live data

Question / action	Primary source/control
HR leave policy	HR policy RAG
Finance sensitive-change policy	Finance policy RAG
Current employee record	Authorized employee database
Current inventory	Operations database
Current finance state	Finance database / authorized enterprise data
Sensitive inventory change	Operations DB + HITL
Sensitive finance change	Finance DB + HITL
Who approved a change?	Audit record/log

12. Implementation Decisions Carried Into the Prototype

Backend modules

- backend/auth — login, JWT and permission checks.
- backend/agents — HR, Operations, intent detection and orchestrator.
- backend/rag — vector store ingestion and secure search.
- backend/hitl — approval creation and approval state.
- backend/actions — approved Finance/Operations database actions.
- backend/audit — audit records/logging.
- backend/database — SQLAlchemy models, initialization and synthetic seed data.
- backend/main.py — protected FastAPI endpoints and orchestration entry point.

Enterprise database model

Model	Purpose
Employee	Identity, company email, password hash and designation.
Department	Department metadata.
InventoryItem	Current inventory quantity, unit, reorder level and supplier link.
Supplier	Supplier identity and supplied item.
FinanceRecord	Synthetic finance records by fiscal year/department/type.
HRRecord	Synthetic employee-related HR records.
ApprovalRecord	Requester, approver, action, target, details, status and timestamps.
AuditRecord	Requester, agent, department, action, details, approver, status and timestamp.

Documents used by RAG

- finance_policy.txt
- hr_policy.txt
- operations_data.txt
- approved_finance_changes.txt
- approved_operations_changes.txt

13. Validation & Evidence from the Prototype

Validation	Result recovered from project work
Intent module compilation	Passed with no output.
Orchestrator compilation	Passed with no output.
Full backend compileall	Passed with no errors.
Frontend production build	Passed with Vite build.
Valid login	Passed.
Invalid login	Passed.
Cross-department query	Passed after department-specific subquery routing.
HITL request	Passed for Operations modification.
GM approval	Passed; approved action executed.
Live inventory	Industrial Valves reached 345 after approved modifications.
Audit logging	Working for sensitive approval/execution flow.

Why the multi-agent fix mattered

An early orchestration implementation sent the complete mixed query to multiple agents. That caused irrelevant cross-domain answers. The implementation was corrected so the orchestrator detects departments, creates department-specific subqueries and then combines only the relevant results. This is an important design lesson from the prototype rather than merely an architecture diagram.

14. Evaluator Workflow

Recommended demonstration

- Login as different synthetic designations and submit the same question to observe RBAC differences.
- Ask an HR policy question to demonstrate RAG.
- Ask a current inventory question to demonstrate live structured data.
- Submit a mixed HR + Operations query to demonstrate multi-agent routing.
- As Operations Admin, request an inventory change.
- As GM, inspect and approve the pending request.
- Repeat the inventory query and show the updated live value.
- Show the audit log proving requester/approver/action/status/time traceability.

Representative queries

What is the HR policy on paid leave?

What does the Finance policy say about sensitive changes?

What is the paid leave policy and what is the current inventory of Industrial Valves?

Increase Industrial Valves by 10 units

Submission package

- GitHub source code.
- README with setup/evaluation instructions.
- PROMPTS.md development prompt record.
- Synthetic data/schema documentation.
- Tests.
- Four-page presentation PDF.
- Optional demonstration video/link.

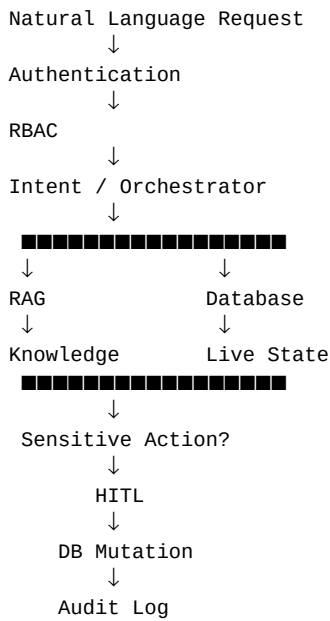
15. Known Boundaries & Future Extensions

- The prompt record is reconstructed, not a verbatim export of private chat history.
- Enterprise data is synthetic/demo data.
- The local prototype uses SQLite; a production deployment could move to a managed relational database without changing the conceptual architecture.
- The prototype focuses on HR, Finance/Enterprise and Operations rather than implementing every possible enterprise department.
- Authorization is intentionally kept outside the LLM so model behavior cannot become the security boundary.
- The architecture is designed to extend to HRIS, ERP, CRM, finance and inventory systems through tools/connectors.

Core architectural lesson

The value of the unified enterprise agent is not the chatbot alone. It is the controlled path from natural language → authorization → relevant knowledge/live state → specialist reasoning → governed action → verification → audit.

Appendix A — Central Prompt & Workflow Pattern



One-line project definition

A role-aware unified enterprise AI layer that combines multi-agent orchestration, permission-aware RAG, live structured data, governed actions, HITL approvals and auditability through one interface.

Source artifacts recovered

- PROMPTS.md — reconstructed development prompt record.
- README.md — comprehensive project documentation.
- KOHLER_README_FINAL.md / architecture variants — concise evaluator documentation.
- Four-page project presentation PDF — problem, approach, technical execution, UX/feasibility and business impact.
- Synthetic enterprise data/schema documentation.
- Submission email template.

End of document.